



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

备忘录模式（Memento）

一、意图（Intent）

在不把模块内部细节暴露给外部的前提下，把某一时刻可恢复的状态保存下来，并在需要时恢复。

站在嵌入式 C 的角度看，备忘录模式控制的不是“对象历史”，而是：

- 某个模块的**可恢复状态**
- 这些状态的**快照边界**
- 快照的**保存时机与恢复时机**
- 快照与模块内部实现之间的**封装边界**

一句话记住它：

模块自己决定哪些状态值得保存，外部只负责触发保存和恢复。

二、动机（Motivation）

在嵌入式系统里，经常有这样一类状态：

- 出厂校准参数
- 用户配置参数
- 通信协议上下文的阶段性状态
- 电机、传感器、显示模块的运行配置

- 升级前需要回滚的关键设置

最直接的做法往往是：

- 业务层直接读取模块内部变量
- 自己拼结构体写入 Flash / NVS / EEPROM
- 恢复时再手工把每个字段写回去

这样的问题很典型：

1. 状态边界会失控

外部代码一旦直接碰模块内部字段，就很容易分不清：

- 哪些字段需要持久化
- 哪些字段只是运行期缓存
- 哪些字段必须成组恢复
- 哪些字段恢复后还要重新 commit

2. 容易破坏模块封装

保存逻辑本来应该知道“哪些状态是有效快照”，结果却变成应用层知道内部布局、知道字段依赖、知道恢复顺序。

3. 非易失存储代价高

嵌入式里写 Flash / NVS 不是零成本操作：

- 有擦写寿命
- 有对齐限制
- 有掉电风险
- 有保存时延
- 还可能受分区、页大小、磨损均衡影响

4. 恢复动作往往不是简单 memcpy

有些参数恢复以后，还需要：

- 重新初始化外设
- 重新装载滤波器系数
- 重新配置通信栈
- 等所有依赖项加载完再统一生效

所以备忘录模式在嵌入式里解决的不是“记录历史”这么抽象的问题，而是：

把“可恢复状态”的定义、导出、恢复，收口到模块自身内部。

三、适用性（Applicability）

下面这些条件出现时，适合考虑备忘录模式：

- 模块内部**有明确状态**，且这些状态需要恢复
- 外部**不能直接依赖内部字段布局**
- 状态保存到非易失介质**代价较高**
- 恢复后可能还要执行**二次生效动作**
- 希望支持“恢复默认值 / 回滚上次配置 / 上电重载参数”

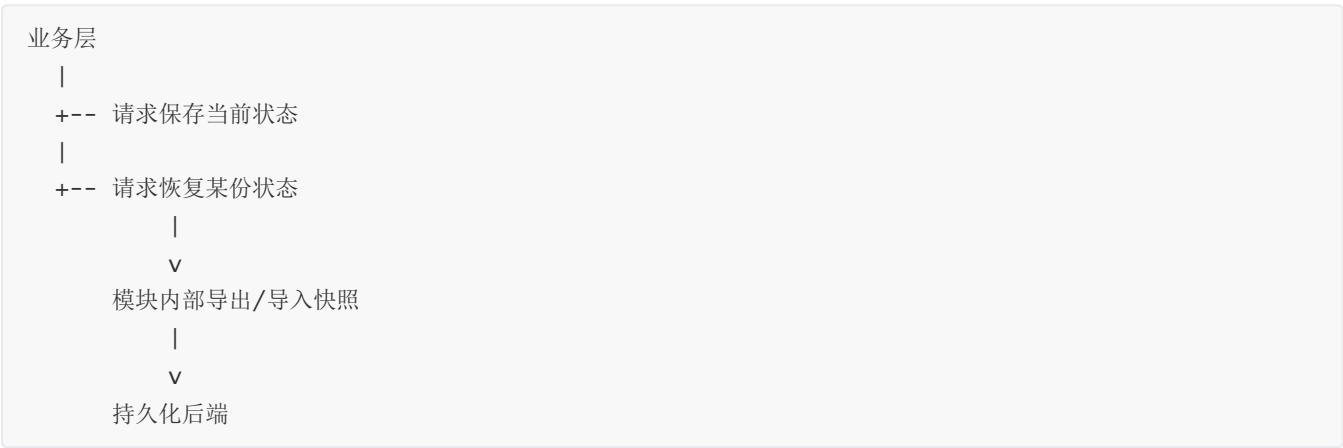
典型嵌入式场景包括：

- 参数子系统
- 校准数据管理
- 升级失败后的配置回滚
- 蓝牙 / Wi-Fi / 网络栈的持久配置
- 复杂外设的 profile 切换与恢复

四、结构（Structure）



也可以按嵌入式流程理解成：



这里最重要的是：

- **业务层不直接改内部布局**

- 快照内容由模块决定
- 存储后端只负责存，不负责理解业务状态

五、参与者（Participants）

1. Originator（发起者 / 状态拥有者）

它是真正持有内部状态的模块。

在嵌入式 C 里，往往就是某个子系统上下文，例如：

- 参数模块上下文
- 电机控制上下文
- 协议会话上下文
- 设备 profile 上下文

它负责：

- 定义哪些状态能进入快照
- 生成快照
- 从快照恢复
- 恢复后做必要的 re-init / commit

2. Memento（备忘录 / 快照）

它不是业务逻辑模块，只是状态载体。

在 C 里通常体现为：

- 一个快照结构体
- 一段序列化 buffer
- 一组键值对导出结果

关键不是“长得像结构体”，而是：

它代表某一时刻的可恢复状态边界。

3. Caretaker（管理者）

它只负责：

- 什么时候保存
- 保存到哪里
- 什么时候恢复
- 保留几份历史快照

它不应该理解模块内部字段语义。

在嵌入式里，通常可以是：

- 参数管理器
- 升级回滚管理器
- 设置子系统
- 配置服务层

六、协作（Collaborations）

整个过程可以拆成两条链：保存链和恢复链。

1. 保存链

1. 业务层判断当前需要保存
2. Caretaker 向模块请求导出快照
3. Originator 生成合法快照
4. Caretaker 把快照写入 Flash / NVS / 文件系统

2. 恢复链

1. 上电或回滚时，Caretaker 从存储介质取回快照
2. 快照交还给 Originator
3. Originator 校验并恢复内部状态
4. 必要时统一 commit，使状态真正生效

最关键的一点是：

是否允许恢复、恢复哪些字段、恢复后还要做什么，都由状态拥有模块说了算。

七、效果（Consequences）

收益

1. 状态边界更清楚

哪些数据是“可恢复状态”，被集中定义，不再散落在应用层。

2. 降低外部耦合

应用层不用知道模块内部字段布局，也不用关心恢复顺序。

3. 更适合持久化与回滚

保存、恢复、恢复默认值、升级回滚，都有统一接口和统一边界。

4. 便于做延迟生效

很多设置不是读到一个字段就立刻生效，而是全部加载完成后统一 commit。

代价

1. 要额外维护快照格式

快照结构、版本兼容、校验字段，都需要设计。

2. 会增加存储与拷贝开销

快照越大，RAM、Flash、时间开销越明显。

3. 版本演进更复杂

字段变化、固件升级、兼容旧数据，都要处理。

八、实现要点 (Implementation)

1. 先划清“哪些状态值得保存”

在嵌入式里，不是所有内部字段都该放进备忘录。

通常要区分：

- **持久配置**：应该保存
- **派生缓存**：恢复后可重算，不必保存
- **瞬时运行态**：通常不保存
- **硬件寄存器镜像**：多半不直接保存，应恢复成配置后再下发

2. 快照格式要带版本与校验

尤其是固件升级会带来结构变化时，要考虑：

- 版本号
- 长度
- CRC / checksum
- 默认值兜底策略

3. 恢复不要等同于裸 memcpy

很多情况下正确顺序应是：

- 先载入原始配置
- 做范围检查
- 解决字段依赖
- 最后统一应用到驱动或硬件

4. 保存策略要考虑写寿命

写 Flash 不能每次字段变化都立刻落盘。

更稳妥的做法往往是：

- dirty 标记
- 定时合并保存
- 关键事件触发保存
- 子树保存而不是全量保存

5. 初始化时机必须明确

恢复逻辑要放在正确阶段：

- 存储后端已就绪
- 依赖的设备已初始化
- 恢复后需要 commit 的模块已经可用

九、典型应用

1. 保存单个配置项或配置子树

在很多实际项目里，并不是每次都全量保存，而是按模块、按 key、按子树保存。Zephyr 文档里提供了 `settings_save_one()`、`settings_save_subtree()` 这类接口，这正符合“快照边界受控”的做法。

2. 上电恢复校准与用户设置

传感器 offset、PID 参数、阈值表、网络凭据，通常都不会让应用层直接拼 Flash 结构体，而是由所属模块自己导出和恢复。

3. 升级回滚前保存旧配置

Bootloader / OTA 管理器在切换镜像之前保存关键配置，失败后再恢复，是备忘录的常见变体。

总结

备忘录模式在嵌入式 C 里，讲的不是“记住过去”，而是把模块自己的可恢复状态封装成可保存、可校验、可回放的快照。